

LOREBOOK

Gestão do Projeto

Backlog priorizado, responsabilidades, evolução e evidências de versionamento.

Integrantes

Pedro Henrique de Araujo Pereira

Henrique Borges da Silva

2026

1. Organização do desenvolvimento

O desenvolvimento do LoreBook foi organizado de forma incremental. As funcionalidades foram implementadas em blocos pequenos, validadas antes da integração e registradas no Git. O GitHub é utilizado como repositório remoto e como histórico das principais evoluções do aplicativo.

A estratégia prioriza primeiro os fluxos centrais da solução - autenticação, biblioteca de obras, capítulos, persistência e permissões - e deixa expansões de infraestrutura e recursos avançados para etapas posteriores. Essa abordagem reduz retrabalho e permite validar cada incremento antes de ampliar o escopo.

2. Distribuição de responsabilidades

Para fins de organização, as responsabilidades são distribuídas por foco principal, mantendo decisões de escopo, validação e revisão final como atividades compartilhadas pelos integrantes.

Responsável	Atividades principais
Pedro Henrique de Araujo Pereira	Implementação técnica; persistência SQLite; autenticação e permissões; arquitetura; integração das funcionalidades; versionamento e organização do repositório.
Henrique Borges da Silva	Revisão de requisitos; validação de fluxos e usabilidade; apoio aos testes; organização da documentação e preparação da apresentação do projeto.
Responsabilidade compartilhada	Priorização do backlog; definição de regras de negócio; revisão funcional; testes de navegação; validação das entregas e decisões de evolução do produto.

3. Backlog priorizado

O backlog abaixo reúne os itens centrais já concluídos e as evoluções planejadas. A prioridade considera impacto sobre o uso do aplicativo e dependência técnica para as etapas seguintes.

Itens concluídos

ID	Item	Prioridade	Situação	Responsável
B01	Navegação principal e protótipo funcional	Alta	Concluído	Compartilhado
B02	Persistência local estruturada com SQLite	Alta	Concluído	Pedro
B03	Cadastro, login, sessão e perfis USER/ADMIN	Alta	Concluído	Pedro
B04	CRUD de obras com proprietário e validações	Alta	Concluído	Pedro
B05	CRUD de capítulos e editor com estados de salvamento	Alta	Concluído	Pedro
B06	Regras de propriedade, permissões e consulta administrativa	Alta	Concluído	Pedro
B07	Feedback, prevenção de erros, acessibilidade e acabamento visual	Média	Concluído	Compartilhado
B08	Reorganização em arquitetura em camadas e Repository Pattern	Média	Concluído	Pedro
B09	Documentação, modelagem e memorial do protótipo	Média	Concluído	Compartilhado

Itens planejados

ID	Item	Prioridade	Situação	Responsável
B10	Persistência remota e sincronização entre dispositivos	Alta	Planejado	Compartilhado
B11	Integração com serviço externo e recursos de IA para apoio ao autor	Alta	Planejado	Compartilhado
B12	Uso de recurso nativo do dispositivo	Média	Planejado	Compartilhado
B13	Persistência completa de personagens, bíblia e notas	Média	Planejado	Compartilhado
B14	Geração de versão instalável e validação em dispositivo físico	Alta	Planejado	Compartilhado

4. Evolução por etapas

Período	Etapas	Resultado
Setembro/2026	Base do aplicativo	Protótipo inicial, navegação e atualização do ambiente Expo.
Outubro/2026	Persistência local	SQLite incorporado para obras e capítulos, com estados de carregamento, erro e salvamento.
Outubro/2026	Contas e autorização	Autenticação local, sessão, perfis USER/ADMIN e propriedade das obras.
Outubro/2026	CRUD e experiência de uso	Criação, edição e exclusão de obras e capítulos, confirmações, feedback e ajustes de acessibilidade.
Outubro/2026	Organização arquitetural	Separação do código em Presentation, Application, Domain, Data e Infrastructure, mantendo Repository Pattern.
Próxima evolução	Serviços conectados	Sincronização remota, serviço externo, recurso nativo e recursos de inteligência artificial de apoio à escrita.

5. Versionamento e rastreabilidade

O código-fonte é versionado com Git e mantido em repositório público no GitHub. As mudanças relevantes são registradas em commits separados, permitindo identificar a sequência de evolução do aplicativo e recuperar estados anteriores quando necessário.

Repositório: github.com/pharaujodev/Prototipo-LoreBook

Data	Commit	Evolução registrada
18/09/2026	6d6d589	Criação inicial do repositório e base do protótipo.
18/09/2026	90faab2	Atualização do ambiente para Expo SDK 57.
08/10/2026	316f716	Persistência SQLite e estados de feedback da interface.
08/10/2026	45d8c6b	Autenticação local, perfis e propriedade das obras.
08/10/2026	6df65e3	CRUD completo, melhorias de usabilidade e acabamento visual.
08/10/2026	f29ec35	Reorganização do código em arquitetura em camadas.

No estado registrado neste documento, a branch principal encontra-se no commit f29ec35. O histórico também contém branches de funcionalidade e de refatoração, usadas para isolar alterações antes da integração com a branch principal.

6. Controle de qualidade

Antes de integrar alterações maiores, o projeto utiliza verificações simples para reduzir regressões e problemas de formatação do código:

- `npm test` - executa os testes automatizados de autenticação, persistência, permissões e CRUD.
- `npm run typecheck` - verifica a tipagem TypeScript sem gerar build.
- `git diff --check` - identifica erros de whitespace antes do commit.
- Exportação web do Expo - utilizada como verificação complementar de empacotamento e compatibilidade do projeto.
- Teste manual dos fluxos principais - cadastro, login, criação de obra, criação/edição de capítulo, exclusão e acesso administrativo.

7. Próximos passos

A continuidade do LoreBook deve preservar a estratégia incremental. A prioridade seguinte é preparar a camada de sincronização remota sem abandonar o SQLite local, seguida da integração com serviços externos e de recursos de inteligência artificial. A IA é prevista como ferramenta de apoio ao autor, podendo auxiliar em revisão, sugestões, organização de ideias e consulta contextual, sem substituir as decisões criativas do usuário.

8. Conclusão

A organização do projeto combina backlog priorizado, divisão de responsabilidades, versionamento e verificações técnicas simples. O histórico do repositório demonstra evolução progressiva do protótipo até a aplicação atual, enquanto o backlog mantém visíveis as funcionalidades já concluídas e as próximas expansões planejadas.